

Hotel Search Application Documentation

Prepared by: Bilal Ahmad Khan, Syed Madiha Husain

Internship at: De Box Global

Project Overview

This project is a web-based hotel search application that allows users to search for hotels in any city using an AI-powered backend. The application leverages the Groq API to generate hotel data dynamically and presents it in an intuitive card-based interface.

Key Features

- **City-based Hotel Search**: Search for hotels in any city worldwide
- **AI-Generated Data**: Uses advanced AI to create realistic hotel information
- **Pagination Support**: Browse through multiple sets of hotel results
- **Responsive Design**: Clean, user-friendly interface with hotel cards
- **Export Functionality**: Built-in support for Excel export (requires implementation)
- **Real-time Loading**: Dynamic content loading with user feedback

Technical Stack

- **Frontend**: HTML5, CSS3, JavaScript (ES6+)
- **API**: Groq API with LLaMA 3 70B model
- **Libraries**: SheetJS (for Excel export functionality)
- **Development**: VS Code with Live Server extension

Code Structure

HTML Structure (`index.html`)

The HTML file creates the foundation of the application:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <!-- Meta tags and styling -->
</head>
<body>
    <!-- Search interface -->
    <!-- Results container -->
    <!-- External library imports -->
```

```
</body>
</html>
```

Key Components:

- **Input Field**: `cityInput` - captures user's city search query
- **Action Buttons**: Search, Next 100 Hotels, Export to Excel
- **Results Container**: `results` - displays hotel cards dynamically
- **External Libraries**: SheetJS for Excel functionality

JavaScript Functionality (script.js)

Core Function: searchHotels(reset = true)

This is the main function that handles hotel searching:

Parameters:

- `reset` (boolean): Determines if this is a new search or pagination

Functionality Flow:

1. **Input Validation**: Checks if city name is provided
2. **State Management**: Manages current city and page number
3. **API Request Construction**: Creates a detailed prompt for the AI model
4. **API Call**: Makes POST request to Groq API endpoint
5. **Response Processing**: Extracts and parses JSON data
6. **UI Update**: Renders hotel cards in the results container

API Integration

The application integrates with Groq's API:

```
const response = await
fetch("https://api.groq.com/openai/v1/chat/completions", {
    method: "POST",
    headers: {
        "Content-Type": "application/json",
```

```
        "Authorization": "Bearer [API_KEY]"
    },
    body: JSON.stringify({
        model: "llama3-70b-8192",
        messages: [{ role: "user", content: prompt }]
    })
});
```

Data Processing

The application processes AI responses through:

1. **JSON Extraction**: Uses regex to extract JSON array from response
2. **Error Handling**: Comprehensive try-catch blocks for API and parsing errors
3. **Data Validation**: Ensures proper JSON structure before rendering

Hotel Data Structure

Each hotel object contains:

```
{
    "name": "string",
    "city": "string",
    "price_per_night": 123,
    "rating": 4.5,
    "amenities": ["string", "string"],
    "available_rooms": 5,
    "address": "string",
    "contact_number": "string",
    "contact_name": "string"
}
```

Setup Instructions

Prerequisites

1. **VS Code Installation**: Download and install Visual Studio Code.
2. **Live Server Extension**: Install the "Live Server" extension.

3. **Internet Connection**: Required for API calls and external library loading.

Installation Steps

1. **Download Repository**:

   https://github.com/bilal-ahmad-khan-07/Project1

   Download the ZIP file and extract it to your desired location.

2. **Open in VS Code**:
   - Launch VS Code
   - File → Open Folder
   - Select the project directory
3. **Project Structure Setup**:
   Ensure your project folder contains:

```
hotel-search-app/
├─── index.html
├─── script.js
```

How to Run the Application

Using Live Server (Recommended)

1. **Open the Project**: Open the project folder in VS Code
2. **Right-click on index.html**: In the file explorer panel
3. **Select "Open with Live Server"**: This will launch the application
4. **Application Launch**: Your default browser will open with the application running at `http://127.0.0.1:5500` (or similar)

Using the Application

Basic Search Process

1. **Enter City Name**: Type the desired city in the input field
2. **Click Search**: Initiates the hotel search for that city
3. **View Results**: Hotel cards will display with complete information
4. **Load More**: Use "Next 100 Hotels" for additional results

Understanding the Results

Each hotel card displays:

- **Hotel Name**: AI-generated hotel name
- **City**: Confirmation of searched city
- **Price per Night**: In Indian Rupees (₹)
- **Rating**: Star rating out of 5
- **Amenities**: List of available facilities
- **Available Rooms**: Current room availability
- **Address**: Full hotel address
- **Contact Information**: Contact person and phone number

Technical Implementation Details

API Communication

The application uses asynchronous JavaScript to communicate with the Groq API:

- **Authentication**: Bearer token authentication
- **Model**: LLaMA 3 70B (llama3-70b-8192)
- **Response Format**: JSON structured data
- **Error Handling**: Multiple layers of error checking

State Management

The application manages state through global variables:

- `currentCity`: Stores the currently searched city
- `currentPage`: Tracks pagination for additional results
- `lastFetchedHotels`: Stores the most recent hotel data

Data Validation

Multiple validation layers ensure data integrity:

1. **Input Validation**: Checks for empty city names
2. **API Response Validation**: Verifies API response structure
3. **JSON Parsing**: Safe JSON parsing with error handling
4. **Content Validation**: Regex-based JSON extraction

Troubleshooting

Common Issues and Solutions

1. **"Please enter a city name" Alert**:
    - **Cause**: Empty input field
    - **Solution**: Enter a valid city name before searching
2. **"Loading..." Persists**:
    - **Cause**: API connection issues or rate limiting
    - **Solution**: Check internet connection, wait and retry
3. **"No valid JSON found in response"**:
    - **Cause**: AI response format issues
    - **Solution**: Retry the search, the AI will generate new data
4. **"Failed to fetch hotels"**:
    - **Cause**: Network connectivity or API key issues
    - **Solution**: Verify internet connection and API key validity

Developer Console Debugging

Enable browser developer tools to view:

- **Console Logs**: API responses and error messages
- **Network Tab**: API request/response details
- **Elements Tab**: DOM structure and styling

Future Enhancements

Planned Features

1. **Excel Export**: Complete implementation of `exportToExcel()` function
2. **Filter Options**: Price range, rating, amenities filtering
3. **Real Hotel Data**: Integration with actual hotel booking APIs
4. **User Preferences**: Save favorite hotels and search history
5. **Responsive Design**: Mobile-optimized interface

Code Improvements

1. **Error Handling**: More granular error messages
2. **Loading States**: Better loading indicators
3. **Caching**: Client-side data caching for better performance
4. **Configuration**: Environment-based API key management

Conclusion

This hotel search application demonstrates modern web development practices using AI integration, responsive design, and user-friendly interfaces. The project showcases skills in HTML, CSS, JavaScript, API integration, and error handling while providing a practical tool for hotel discovery.

The application serves as a foundation for more complex hotel booking systems and demonstrates the potential of AI-powered data generation for prototyping and development purposes.